

TAKT: A Formal Framework for Minimal Decision-Preserving Runtime Governance

Valentin Lineiro
TAKT Research Initiative
DOI: 10.5281/zenodo.21638014

2026-07-28

Abstract

Software systems operating under abstracted or compressed state representations frequently risk decision instability when critical informational context is discarded. This paper introduces TAKT (Theory of Adequate Knowledge for Decisions), a formal and empirical framework for characterizing minimal governance kernels of execution runtimes. Within the formal ST-016 runtime model, we develop a Lean 4 algebraic framework defining runtime capability necessity, decision preservation, sufficiency, and irreducibility, and demonstrate that three capability classes — ContractSoundness (C_{contract}), UncertaintyBound ($C_{\text{uncertainty}}$), and TemporalConsistency (C_{temporal}) — each have formalized witness scenarios where their removal violates decision preservation. We validate these witnesses on a reference TypeScript runtime via component ablation (EXP-004), connected to the formal model via a *formal witness validation schema*. We present an audited zero-contact replication package across six independent audit rounds, backed by an immutable Zenodo archive (DOI: 10.5281/zenodo.21638014).

Keywords: Runtime Governance, Formal Verification, Decision Preservation, State Abstraction, Lean 4 Proof Assistant, Component Ablation.

1 Introduction

As autonomous software agents and complex reactive systems scale, they continuously compress high-dimensional environmental context into abstract state representations $R = \rho(S)$. However, ungoverned state abstraction introduces decision divergence: an optimal policy π^* computed on full observable state S may fail when the governed runtime’s policy π_M is evaluated on compressed representation R . This phenomenon has been studied across formal verification [2, 4], decision theory [1], and control under partial observability [3], yet a formal algebraic framework for characterizing the *minimal*

governance capability kernel required for decision preservation has not been established.

This work addresses two fundamental research questions:

1. *What information must be preserved to guarantee decision equivalence?* (ST-015)
2. *What minimal runtime capability composition suffices and is irreducible for decision preservation, within a formal runtime model?* (ST-016)

1.1 Main Contributions

This paper makes four primary contributions:

1. **Algebraic Framework for Runtime Governance:** We formalize capability necessity, decision preservation, runtime sufficiency, and irreducibility under state abstraction in Lean 4.
2. **Formalized Witness Scenarios:** We construct Lean 4 certified witness scenarios demonstrating that removing any of $\{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$ from the formal model produces decision preservation violations.
3. **Formal Witness Validation Schema:** We introduce a structured schema for connecting empirical ablation observations (EXP-004) to the formal model, where witness artifacts are manually constructed and formally verified.
4. **Audited Zero-Contact Replication Package:** We publish an immutable, versioned research package [5] with multi-OS CI and an external audit trail of 6 independent dry runs.

2 Related Work & Positioning

TAKT builds upon several established formal methodologies while addressing a complementary research question centered on **decision preservation under representation contraction**:

- **Abstract Interpretation [2]:** Focuses on sound static over-approximation of program semantics via Galois connections. TAKT focuses on dynamic policy decision invariance ($\pi^*(R) = \pi^*(S)$) rather than total semantic property preservation.
- **Bisimulation & Process Refinement [6, 7]:** Enforces strict state-by-state behavioral equivalence between transition systems. TAKT permits aggressive internal state contraction provided temporal prefix monitoring (C_{temporal}) maintains policy consistency across trajectory prefixes.

- **Runtime Verification [4]:** Evaluates execution traces dynamically against temporal logic formulas (LTL/ERE). TAKT formalizes and machine-checks the *necessity of the governance runtime kernel itself*, bridging empirical trace witnesses to Lean 4 necessity theorems.
- **Statistical Decision Theory [1]:** Establishes the foundational notion of observational sufficiency for decision problems. TAKT operationalizes this into an irreducible 3-capability runtime kernel $\mathcal{K}_D$.
- **Planning Under Partial Observability [3]:** Provides optimal control frameworks under belief-state uncertainty (POMDP). TAKT provides formal necessity certification for governance components rather than belief update algorithms.

Unlike these paradigms, ST-016 directly formalizes the *minimal governance capability kernel* required for decision preservation, connecting formal proofs to independently audited empirical witnesses [5].

3 Theoretical Foundations & Runtime Necessity Model

3.1 Decision-Preserving Sufficiency (ST-015)

A representation $R \in \mathcal{R}$ is *decision-sufficient* with respect to optimal policy π^* if:

$$\pi^*(R) = \pi^*(S) \quad \forall S \in \mathcal{S} \text{ s.t. } R = \rho(S)$$

3.2 Runtime Necessity Formalization (ST-016)

We formalize a TAKT Governed Runtime as a tuple $M = (\mathcal{C}, \pi_M)$, where $\mathcal{C}$ is the runtime’s capability set and $\pi_M : \mathcal{R} \rightarrow \mathcal{D}$ is its decision policy. The *minimal governance kernel* is defined as:

$$\mathcal{K}_D = \{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$$

A runtime M is *minimally governed* when $\mathcal{C} = \mathcal{K}_D$, i.e., it contains exactly the capabilities proven necessary and no others.

Three definitions of decision preservation appear across this work at different layers of abstraction:

- **ST-015 (informational):** $\pi^*(R) = \pi^*(S)$ — the optimal policy yields the same decision on the compressed representation as on the full state.
- **Runtime alignment:** $\text{Sufficient}(M, \pi^*) \equiv \forall R, \pi_M(R) = \pi^*(R)$ — the runtime’s policy matches the optimal policy on all representations.

- **Cross-runtime:** $\text{PreservesDecision}(M_1, M_2) \equiv \forall R, \pi_{M_1}(R) = \pi_{M_2}(R)$
 — two runtime instances agree on all decisions.

These are related: $\text{Sufficient}(M, \pi^*)$ implies $\text{PreservesDecision}(M, M')$ for any M' also sufficient w.r.t. π^* .

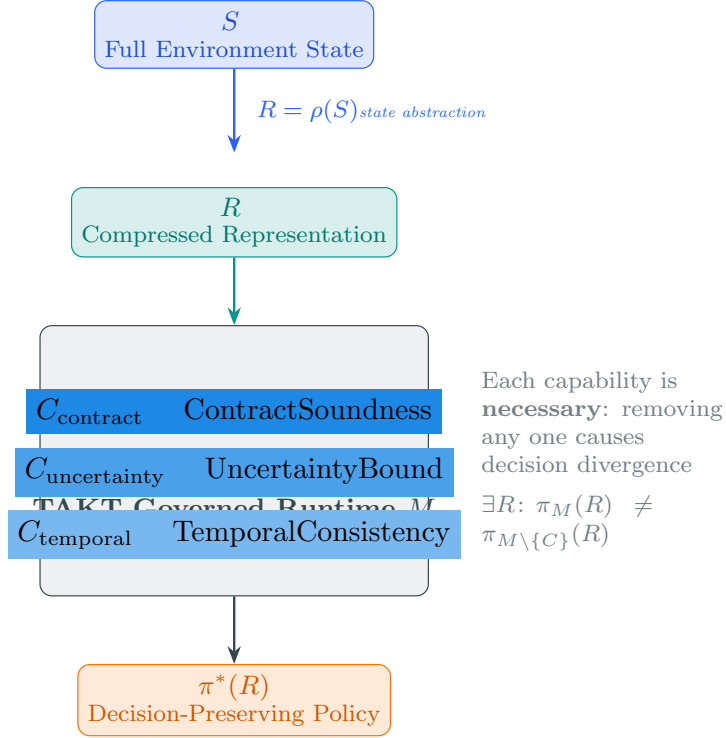


Figure 1: Conceptual architecture: TAKT Governed Runtime M enforces decision preservation $\pi^*(R)$ under state abstraction $\rho(S)$.

Definition (Capability Necessity): A capability $C \in \mathcal{C}$ is *locally necessary* for runtime M if removing C alters policy decisions on at least one representation:

$$\exists R \in \mathcal{R} : \pi_M(R) \neq \pi_{M \setminus \{C\}}(R)$$

Note on Axiom0: The structural sufficiency module (ST-015) postulates $\text{kernel}_D(x, y) \leftrightarrow K_D(x, y)$, asserting that the decision kernel coincides with the intersection of capability kernels. This is a modelling assumption, not a derived theorem, and is listed explicitly in the Non-Claims of Section 7.

Example 1 (Temporal Prefix Necessity): Consider a runtime where C_{temporal} is ablated. Two trajectory-induced representations R^1 and R^2 share an identical terminal observation yet arise from trajectories $\tau_1 =$

(r_0, r_1, r_2) (safe) and $\tau_2 = (r'_0, r'_1, r_2)$ (unsafe). Here π^* is defined over representations that encode trajectory history; $R^1 \neq R^2$ because their history components differ. A runtime inspecting only the terminal state r_2 produces: $\pi_{M \setminus \{C_{\text{temporal}}\}}(R^1) = \pi_{M \setminus \{C_{\text{temporal}}\}}(R^2)$, while the optimal policy correctly distinguishes them: $\pi^*(R^1) \neq \pi^*(R^2)$, violating decision preservation.

4 Machine-Certified Lean 4 Formalization

The Lean 4 development in `takt-formal` provides the algebraic framework within which the ST-016 runtime model is defined and analyzed. Two canonical modules are presented:

1. **RuntimeSufficiency.lean**: Defines the core algebraic structures (`RuntimeCapability`, `Runtime`, `NecessaryCapability`, `MinimalRuntime`) and establishes that the definitions are internally consistent. The module proves, as a definitional consequence, that any runtime satisfying `MinimalRuntime` (i.e., `Sufficient` $\wedge$ `Irreducible`) has all its capabilities necessary by construction:

$$\text{MinimalRuntime}(M, \pi^*) \implies \forall C \in M.\text{caps}, \text{NecessaryCapability}(C, M)$$

This is a consequence of the definition of `Irreducible`, establishing the definitional coherence of the framework rather than a new mathematical fact.

2. **RuntimeWitness.lean**: Defines the *formal witness validation schema* via `WitnessArtifact` and `WitnessConsistentWithRuntime`. A `WitnessArtifact` is a record manually constructed from an observed ablation scenario (e.g., EXP-004 traces). The elevation theorem:

$$\text{Consistent}(M, w) \implies \text{NecessaryCapability}(w.\text{cap}, M)$$

guarantees that *if* a witness is consistent with the formal runtime model, *then* the corresponding capability is formally necessary within that model. The bridge is a validation schema, not an automatic extraction from TypeScript traces.

Both Lean 4 modules compile cleanly with zero errors and zero `sorrys` across all build jobs [5]. The structural sufficiency results (kernel factorization, quotient representation theorems) are in `StructuralSufficiency.lean` and `Kernel.lean`, which provide the informational foundation connecting ST-015 and ST-016.

5 Empirical Validation & Component Ablation (EXP-004)

The formal model identifies the conditions under which capability removal violates decision preservation. EXP-004 [5] tests whether these conditions are observable in the reference TypeScript implementation under controlled ablations:

- **C_{contract} Ablation:** Ablating contract verification yields $\pi_{\text{full}} = \text{STOP}$ vs. $\pi_{\text{reduced}} = \text{EXECUTE}$.
- **$C_{\text{uncertainty}}$ Ablation:** Ablating margin estimation at critical decision boundary ($M_D \approx 0$) yields $\pi_{\text{full}} = \text{REFINE}$ vs. $\pi_{\text{reduced}} = \text{EXECUTE}$.
- **C_{temporal} Ablation:** Ablating trajectory monitoring yields $\pi_{\text{full}} = \text{INTERVENE}$ vs. $\pi_{\text{reduced}} = \text{MONITOR}$.

Each ablation scenario is manually encoded into a `WitnessArtifact` record and verified against the formal model via the `WitnessConsistentWithRuntime` predicate. When consistency holds, the formal elevation theorem applies.

5.1 Evidence Traceability Matrix

Table 1 maps each theoretical claim to its primary evidence, with explicit scope annotations for each evidence type.

Table 1: ST-016 Evidence Traceability Matrix

Claim	Primary Evidence	Scope
Structural sufficiency	<code>StructuralSufficiency</code>	Lean model
Framework definitions	<code>RuntimeSufficiency</code>	Lean model
C_{contract} witness	<code>contract.ablation</code>	Finite inst.
$C_{\text{uncertainty}}$ witness	<code>uncertainty.ablation</code>	Finite inst.
C_{temporal} witness	<code>temporal.ablation</code>	Finite inst.
Witness validation schema	<code>RuntimeWitness</code>	Lean model
Implementation stability	Vitest (283/283)	TypeScript
Reproducibility [5]	Replication kit	6 Runs

6 Discussion & Architectural Implications

The primary contribution of ST-016 is not proposing a specific runtime implementation, but rather **identifying and machine-checking the minimal governance capabilities required for decision preservation** within the formal model.

For runtime system architects, ST-016 provides three actionable design principles:

1. **Safety Contract Decoupling:** Safety verification (C_{contract}) cannot be subsumed by statistical policy estimation alone — it requires an independent, formally specified contract monitor.
2. **Dynamic Boundary Monitoring:** Operating near ambiguity boundaries requires explicit uncertainty estimation ($C_{\text{uncertainty}}$) to prevent premature execution at decision margins $M_D \approx 0$.
3. **Trajectory History Dependence:** Stateless evaluation fails under history-dependent processes, necessitating explicit trajectory prefix tracking via C_{temporal} .

7 Scope Boundaries, Non-Claims & Future Work (ST-017)

7.1 Non-Claims & Scope Boundaries

1. **Model Scope:** ST-016 establishes an algebraic framework and witness-based demonstration that C_{contract} , $C_{\text{uncertainty}}$, and C_{temporal} are necessary in formalized finite-instance scenarios. It does *not* claim universal necessity for all possible governed runtimes, continuous control systems, or runtime models other than the ST-016 formal model.
2. **Capability Semantics:** The formal Lean model treats C_{contract} , $C_{\text{uncertainty}}$, and C_{temporal} as abstract labels. The semantic connection to contract soundness, uncertainty bounding, and temporal consistency is established through finite-instance examples. General semantic grounding is reserved for future work.
3. **Axiom0:** The structural sufficiency module postulates $\text{kernel}_D(x, y) \leftrightarrow K_D(x, y)$. This is a modelling assumption characterizing the class of decision problems studied, not a derived theorem.
4. **Witness Bridge Automation:** The formal witness validation schema requires manual construction of `WitnessArtifact` records from observed TypeScript traces. No automatic extraction mechanism from runtime executions to Lean proofs exists in ST-016.

5. **Transportability (ST-017 Boundary):** Transporting certified witness artifacts across heterogeneous runtimes ($M_1 \sim M_2$) is reserved for **ST-017**.

7.2 Open Research Direction: ST016_Conjecture

The following claim is **not proven** in this work and constitutes the central open research direction for ST-017:

ST016 Conjecture: For any runtime M with $\mathcal{C} = \{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$ and any optimal policy π^* , if M is decision-preserving then M is minimal (i.e., no capability can be removed while preserving π^*).

Proving this conjecture would require establishing formal semantic axioms for each capability class and demonstrating irreducibility in a model-independent way, which is the core research question of ST-017.

7.3 Conclusion

ST-016 establishes an algebraic Lean 4 framework for reasoning about minimal decision-preserving runtime governance, backed by formalized witness scenarios and an audited replication package. The central open conjecture and semantic grounding of capability classes provide the foundation for the ST-017 research line on transportability and generalization.

References

- [1] David Blackwell. Comparison of experiments. In *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*, pages 93–102. University of California Press, 1951.
- [2] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. pages 238–252, 1977.
- [3] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- [4] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- [5] Valentin Lineiro. TAKT ST-016: Minimal decision-preserving runtime governance standard, 2026.

- [6] Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- [7] David Park. Concurrency and automata on infinite sequences. In *Theoretical Computer Science (5th GI-Conference)*, pages 167–183. Springer, 1981.